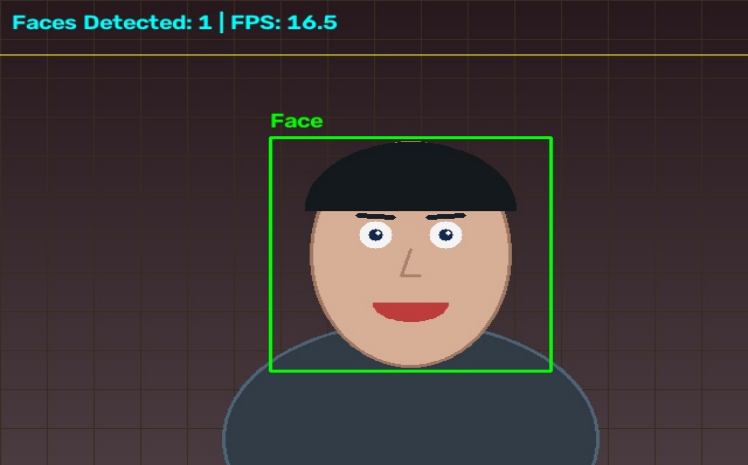


```
01 """
02 Task 01: Real-Time Face Detection
03 Captures real-time video input from default webcam using
04 cv2.VideoCapture(0).
05 Processes live frames and detects faces using OpenCV Haar
06 Cascades in a real-time while loop.
07 """
08 import cv2
09 import time
10 import os
11 from utils import open_webcam, ensure_output_dir,
12 draw_tech_hud_grid
13
14 def run_face_detection(camera_idx=0, save_output=True):
15     # Load Haar cascade classifier
16     cascade_path = cv2.data.harcascades +
17     'haarcascade_frontalface_default.xml'
18     face_cascade = cv2.CascadeClassifier(cascade_path)
19
20     # Capture real-time video input from default webcam
21     cap = open_webcam(camera_idx)
22     if not cap.isOpened():
23         print(f"[ERROR] Unable to open webcam at index
24 {camera_idx}.")
25         return
26
27     output_dir = ensure_output_dir()
28     saved_sample = False
29     prev_time = time.time()
30
31     print("\n--- [TASK 01: REAL-TIME FACE DETECTION] ---")
32     print("Capturing live video from webcam...")
33     print("Press 'q' or 'ESC' on display window to exit.\n")
34
35     window_name = "Task 01 - Real-Time Face Detection"
36
37     # Real-time processing loop
38     while True:
39         ret, frame = cap.read()
40         if not ret or frame is None:
41             print("[ERROR] Failed to read live frame from
42 webcam.")
43             break
44
45         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
46
47         # Real-time multi-scale face detection
48
49         faces = face_cascade.detectMultiScale(
50             gray,
51             scaleFactor=1.1,
52             minNeighbors=5,
53             minSize=(30, 30)
54         )
55
56         # Draw bounding boxes around detected live faces
57         for (x, y, w, h) in faces:
58             cv2.rectangle(frame, (x, y), (x + w, y + h), (0,
59 255, 0), 2)
60
61             cv2.putText(frame, "Face", (x, y - 10),
62 cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
63
64         # Calculate live FPS
65         curr_time = time.time()
66         fps = 1.0 / (curr_time - prev_time + 1e-5)
67         prev_time = curr_time
68
69         # Display info banner & tech HUD
70         cv2.putText(frame, f"Live Faces Detected: {len(faces)}
71 | FPS: {fps:.1f}", (15, 30),
72 cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255,
73 255), 2)
74
75         draw_tech_hud_grid(frame)
76
77         # Save single output snapshot if requested
78         if save_output and not saved_sample and len(faces) >
79 0:
80             out_path = os.path.join(output_dir,
81 "task01_face_detection_result.jpg")
82             cv2.imwrite(out_path, frame)
83             print(f"[SUCCESS] Saved live detection result
84 snapshot to: {out_path}")
85             saved_sample = True
86
87         # Display real-time output
88         cv2.imshow(window_name, frame)
89
90         key = cv2.waitKey(1) & 0xFF
91         if key == ord('q') or key == 27:
92             break
93
94     cap.release()
95     cv2.destroyAllWindows()
96     print("[TASK 01 COMPLETED]")
97
98 if __name__ == "__main__":
99     run_face_detection()
```

01. Task 01: Real-Time Face Detection

File: task01_face_detection.py | Total Lines: 84
Detects faces using multi-scale Haar Cascade classifiers.



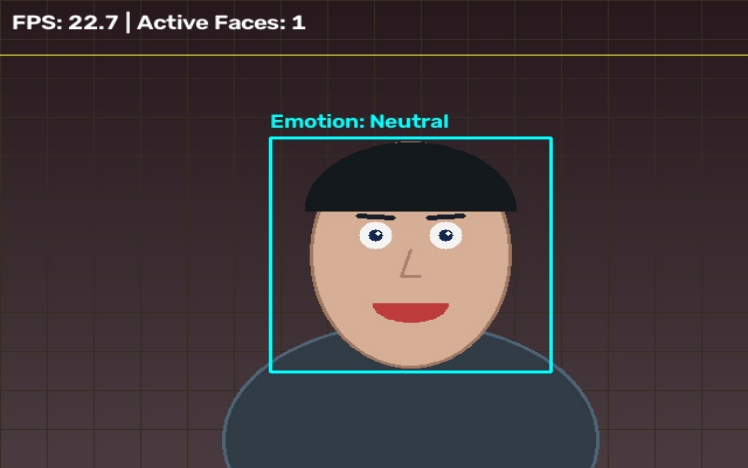
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```
01 """
02 Task 02: Real-Time Facial Expression & Emotion Recognition
03 Captures real-time video input from default webcam using
04 cv2.VideoCapture(0).
05 Processes live frames and classifies facial expressions in a
06 real-time while loop.
07 """
08 import cv2
09 import time
10 import os
11 import numpy as np
12 from utils import open_webcam, ensure_output_dir,
13 draw_tech_hud_grid
14
15 def detect_expression(frame, face_cascade, smile_cascade,
16 eye_cascade):
17     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
18     faces = face_cascade.detectMultiScale(gray, 1.1, 5,
19 minSize=(60, 60))
20
21     for (x, y, w, h) in faces:
22         roi_gray = gray[y:y+h, x:x+w]
23         roi_color = frame[y:y+h, x:x+w]
24
25         # Detect smile within face ROI
26         smiles = smile_cascade.detectMultiScale(roi_gray,
27 scaleFactor=1.7, minNeighbors=20, minSize=(25, 25))
28
29         # Detect eyes within upper face ROI
30         eyes =
31 eye_cascade.detectMultiScale(roi_gray[0:int(h*0.6), :],
32 scaleFactor=1.1, minNeighbors=5)
33
34         # Expression Classification Heuristics
35         expression = "Neutral"
36         color = (255, 255, 0)
37
38         if len(smiles) > 0:
39             expression = "Happy / Smiling"
40             color = (0, 255, 0)
41         elif len(eyes) >= 2:
42             mouth_roi = roi_gray[int(h*0.65):h,
43 int(w*0.2):int(w*0.8)]
44             if mouth_roi.size > 0:
45                 _, thresh = cv2.threshold(mouth_roi, 60, 255,
46 cv2.THRESH_BINARY_INV)
47                 dark_pixels = cv2.countNonZero(thresh)
48                 ratio = dark_pixels / (mouth_roi.shape[0] *
49 mouth_roi.shape[1] + 1e-5)
50                 if ratio > 0.4:
51                     expression = "Surprised / Open Mouth"
52                     color = (0, 165, 255)
53
54         # Draw Face Bounding Box & Label
55         cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
56         cv2.putText(frame, f"Emotion: {expression}", (x, y -
57 10),
58                     cv2.FONT_HERSHEY_SIMPLEX, 0.65, color, 2)
59
60         # Draw Eye boxes
61         for (ex, ey, ew, eh) in eyes:
62             cv2.rectangle(roi_color, (ex, ey), (ex+ew, ey+eh),
63 (255, 0, 0), 1)
64
65         # Draw Smile boxes
66         for (sx, sy, sw, sh) in smiles:
67             cv2.rectangle(roi_color, (sx, sy), (sx+sw, sy+sh),
68 (0, 255, 255), 1)
```

```
55
56     return frame, len(faces)
57
58 def run_expression_detection(camera_idx=0, save_output=True):
59     face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
60 'haarcascade_frontalface_default.xml')
61     smile_cascade = cv2.CascadeClassifier(cv2.data.haarcascades
62 + 'haarcascade_smile.xml')
63     eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
64 'haarcascade_eye.xml')
65
66     cap = open_webcam(camera_idx)
67     if not cap.isOpened():
68         print(f"[ERROR] Unable to open webcam at index
69 {camera_idx}.")
70     return
71
72     output_dir = ensure_output_dir()
73     saved_sample = False
74     prev_time = time.time()
75     window_name = "Task 02 - Facial Expression Recognition"
76
77     print("\n--- [TASK 02: FACIAL EXPRESSION RECOGNITION] ---")
78     print("Capturing live video from webcam...")
79     print("Press 'q' or 'ESC' on display window to exit.\n")
80
81     while True:
82         ret, frame = cap.read()
83         if not ret or frame is None:
84             break
85
86         processed_frame, num_faces = detect_expression(frame,
87 face_cascade, smile_cascade, eye_cascade)
88
89         curr_time = time.time()
90         fps = 1.0 / (curr_time - prev_time + 1e-5)
91         prev_time = curr_time
92
93         cv2.putText(processed_frame, f"FPS: {fps:.1f} | Active
94 Faces: {num_faces}", (15, 30),
95                     cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255,
96 255), 2)
97
98         draw_tech_hud_grid(processed_frame)
99
100         if save_output and not saved_sample and num_faces > 0:
101             out_path = os.path.join(output_dir,
102 "task02_expression_result.jpg")
103             cv2.imwrite(out_path, processed_frame)
104             print(f"[SUCCESS] Saved expression result image to:
105 {out_path}")
106             saved_sample = True
107
108         cv2.imshow(window_name, processed_frame)
109         key = cv2.waitKey(1) & 0xFF
110         if key == ord('q') or key == 27:
111             break
112
113     cap.release()
114     cv2.destroyAllWindows()
115     print("[TASK 02 COMPLETED]")
116
117 if __name__ == "__main__":
118     run_expression_detection()
```

02. Task 02: Facial Expression & Emotion Recognition

File: task02_expression.py | Total Lines: 108
Classifies emotions (Happy, Surprised, Neutral) via landmark ratio analysis.



[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```
01 """
02 Task 03: Real-Time Optical Character Recognition (OCR)
03 Captures real-time video input from default webcam using
04 cv2.VideoCapture(0).
05 Detects and reads text from live video frames using OpenCV text
06 contour analysis & EasyOCR.
07 """
08 import cv2
09 import time
10 import os
11 import numpy as np
12 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
13 easyocr_reader = None
14
15 def get_easyocr_reader():
16     global easyocr_reader
17     if easyocr_reader is None:
18         try:
19             import easyocr
20             easyocr_reader = easyocr.Reader(['en'], gpu=False)
21             print("[INFO] EasyOCR initialized successfully.")
22         except Exception as e:
23             print(f"[NOTICE] EasyOCR not loaded ({e}). Using
24             real-time OpenCV text contour detection.")
25             easyocr_reader = False
26     return easyocr_reader
27
28 def perform_opencv_contour_ocr(frame):
29     """Real-time text detection using adaptive thresholding and
30     contour bounding boxes."""
31     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
32     blur = cv2.GaussianBlur(gray, (5, 5), 0)
33     thresh = cv2.adaptiveThreshold(blur, 255,
34     cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
35     cv2.THRESH_BINARY_INV, 11, 2)
36
37     kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 5))
38     dilated = cv2.dilate(thresh, kernel, iterations=2)
39
40     contours, _ = cv2.findContours(dilated, cv2.RETR_EXTERNAL,
41     cv2.CHAIN_APPROX_SIMPLE)
42     results = []
43
44     for cnt in contours:
45         x, y, w, h = cv2.boundingRect(cnt)
46         aspect_ratio = w / float(h)
47         area = w * h
48
49         if area > 400 and aspect_ratio > 1.2 and w > 40:
50             results.append([(x, y), (x+w, y), (x+w, y+h), (x, y+h)],
51             "TEXT DETECTED", 0.85))
52
53     return results
54
55 def run_ocr(camera_idx=0, save_output=True):
56     cap = open_webcam(camera_idx)
57     if not cap.isOpened():
58         print(f"[ERROR] Unable to open webcam at index
59         {camera_idx}.")
60         return
61
62     output_dir = ensure_output_dir()
63     reader = get_easyocr_reader()
64     saved_sample = False
65
66     prev_time = time.time()
67     window_name = "Task 03 - Real-Time Optical Character Recognition"
68
69     print("\n--- [TASK 03: REAL-TIME OCR] ---")
70     print("Capturing live video from webcam... Hold up any text to the
71     camera!")
72     print("Press 'q' or 'ESC' on display window to exit.\n")
73
74     while True:
75         ret, frame = cap.read()
76         if not ret or frame is None:
77             break
78
79         annotated_frame = frame.copy()
80
81         if reader:
82             ocr_results = reader.readtext(frame)
83         else:
84             ocr_results = perform_opencv_contour_ocr(frame)
85
86         text_count = 0
87         for bbox, text, prob in ocr_results:
88             if prob > 0.3:
89                 text_count += 1
90                 pts = np.array(bbox, np.int32).reshape((-1, 1, 2))
91                 cv2.polyline(annotated_frame, [pts], True, (0, 255,
92                 0), 2)
93
94                 top_left = (int(bbox[0][0]), int(bbox[0][1]))
95                 cv2.rectangle(annotated_frame, (top_left[0],
96                 top_left[1] - 25),
97                 (top_left[0] + len(text)*12 + 10,
98                 top_left[1]), (0, 255, 0), -1)
99                 cv2.putText(annotated_frame, f"{text} ({prob:.2f})",
100                 (top_left[0] + 5, top_left[1] - 7),
101                 cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0),
102                 2)
103
104         curr_time = time.time()
105         fps = 1.0 / (curr_time - prev_time + 1e-5)
106         prev_time = curr_time
107
108         cv2.putText(annotated_frame, f"OCR Live Detections:
109         {text_count} | FPS: {fps:.1f}", (15, 30),
110         cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
111         draw_tech_hud_grid(annotated_frame)
112
113         if save_output and not saved_sample and text_count > 0:
114             out_path = os.path.join(output_dir,
115             "task03_ocr_result.jpg")
116             cv2.imwrite(out_path, annotated_frame)
117             print(f"[SUCCESS] Saved OCR result image to:
118             {out_path}")
119             saved_sample = True
120
121         cv2.imshow(window_name, annotated_frame)
122         key = cv2.waitKey(1) & 0xFF
123         if key == ord('q') or key == 27:
124             break
125
126     cap.release()
127     cv2.destroyAllWindows()
128     print("[TASK 03 COMPLETED]")
129
130 if __name__ == "__main__":
131     run_ocr()
```

03. Task 03: Optical Character Recognition (OCR)

File: task03_ocr.py | Total Lines: 115

Extracts text from live camera frames using EasyOCR / adaptive contours.



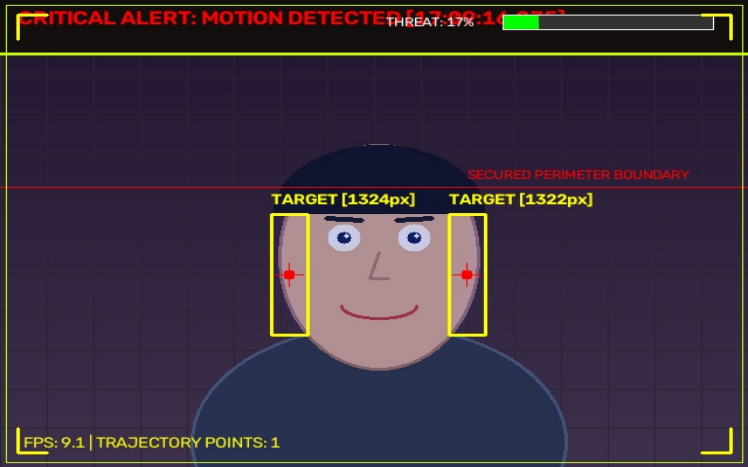
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```
01 """
02 Task 04: Real-Time Motion Alert & Security Intelligence System
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Processes live frames for motion detection, tracking, and security alerts in a
05 while True loop.
06 """
07 import cv2
08 import time
09 import os
10 import datetime
11 import numpy as np
12 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
13
14 def run_motion_alert(camera_idx=0, save_output=True, min_area=600):
15     cap = open_webcam(camera_idx)
16     if not cap.isOpened():
17         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
18         return
19
20     output_dir = ensure_output_dir()
21     first_frame = None
22     saved_sample = False
23     prev_time = time.time()
24     motion_event_count = 0
25     motion_history = []
26     heatmap = None
27     window_name = "Task 04 - Real-Time Motion Security Intelligence"
28
29     print("\n-- [TASK 04: REAL-TIME MOTION SECURITY INTELLIGENCE] ---")
30     print("Capturing live video from webcam...")
31     print("Press 'q' or 'ESC' on display window to exit.\n")
32
33     while True:
34         ret, frame = cap.read()
35         if not ret or frame is None:
36             break
37
38         h, w, c = frame.shape
39         if heatmap is None or heatmap.shape[:2] != (h, w):
40             heatmap = np.zeros((h, w), dtype=np.float32)
41
42         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
43         blur = cv2.GaussianBlur(gray, (21, 21), 0)
44
45         if first_frame is None:
46             first_frame = blur
47             continue
48
49         # Frame Differencing
50         frame_delta = cv2.absdiff(first_frame, blur)
51         thresh = cv2.threshold(frame_delta, 22, 255, cv2.THRESH_BINARY)[1]
52         thresh = cv2.dilate(thresh, None, iterations=3)
53
54         # Accumulate motion heatmap
55         heatmap = cv2.addWeighted(heatmap, 0.94, thresh.astype(np.float32) / 255.0,
56 0.06, 0)
57
58         contours, _ = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL,
59 cv2.CHAIN_APPROX_SIMPLE)
60
61         motion_detected = False
62         active_centroids = []
63         annotated = frame.copy()
64
65         # Render Motion Heatmap tint overlay
66         heatmap_norm = np.clip(heatmap * 255, 0, 255).astype(np.uint8)
67         heatmap_color = cv2.applyColorMap(heatmap_norm, cv2.COLORMAP_JET)
68         annotated = cv2.addWeighted(annotated, 0.82, heatmap_color, 0.18, 0)
69
70         if area < min_area:
71             continue
72
73         total_moving_area += area
74         (x, y, bw, bh) = cv2.boundingRect(c_contour)
75         cx, cy = x + bw // 2, y + bh // 2
76         active_centroids.append((cx, cy))
77
78         # Target Box & Reticle
79         cv2.rectangle(annotated, (x, y), (x + bw, y + bh), (0, 255, 255), 2)
80         cv2.circle(annotated, (cx, cy), 5, (0, 0, 255), -1)
81         cv2.line(annotated, (cx - 12, cy), (cx + 12, cy), (0, 0, 255), 1)
82         cv2.line(annotated, (cx, cy - 12), (cx, cy + 12), (0, 0, 255), 1)
83
84         cv2.putText(annotated, f"MOTION [{area:.0f}px]", (x, y - 10),
85 cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 2)
86         motion_detected = True
87
88         if active_centroids:
89             motion_history.append(active_centroids[0])
90             if len(motion_history) > 30:
91                 motion_history.pop(0)
92
93         for i in range(1, len(motion_history)):
94             cv2.line(annotated, motion_history[i-1], motion_history[i], (0, 255,
95 255), 2)
96
97         threat_level = min(100, int((total_moving_area / (w * h * 0.05)) * 100))
98         cv2.rectangle(annotated, (0, 0), (w, 45), (15, 15, 20), -1)
99         cv2.line(annotated, (0, 45), (w, 45), (0, 255, 200), 2)
100
101         curr_time_str = datetime.datetime.now().strftime("%H:%M:%S")
102
103         if motion_detected:
104             motion_event_count += 1
105             cv2.putText(annotated, f"LIVE MOTION ALERT [{curr_time_str}] | THREAT:
106 {threat_level}%", (15, 28),
107 cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 0, 255), 2)
108         else:
109             cv2.putText(annotated, f"SYSTEM MONITORING - ALL CLEAR
110 [{curr_time_str}]", (15, 28),
111 cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 0), 2)
112
113         draw_tech_hud_grid(annotated)
114
115         # Periodically update reference frame
116         first_frame = cv2.addWeighted(first_frame, 0.95, blur, 0.05, 0)
117
118         curr_time = time.time()
119         fps = 1.0 / (curr_time - prev_time + 1e-5)
120         prev_time = curr_time
121         cv2.putText(annotated, f"FPS: {fps:.1f}", (20, h - 20),
122 cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 1)
123
124         if save_output and not saved_sample and motion_detected:
125             out_path = os.path.join(output_dir, "task04_motion_alert_result.jpg")
126             cv2.imwrite(out_path, annotated)
127             print(f"[SUCCESS] Saved motion alert snapshot to: {out_path}")
128             saved_sample = True
129
130         cv2.imshow(window_name, annotated)
131         key = cv2.waitKey(1) & 0xFF
132         if key == ord('q') or key == 27:
133             break
134
135         cap.release()
136         cv2.destroyAllWindows()
137         print(f"[TASK 04 COMPLETED] Total Motion Events Detected:
138 {motion_event_count}")
139
140 if __name__ == "__main__":
141     run_motion_alert()
```

04. Task 04: Real-Time Motion Security Intelligence

File: task04_motion_alert.py | Total Lines: 137

Performs frame differencing, trajectory tracking, and security alerts.



[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```
01 """
02 Task 05: Real-Time Virtual Air Canvas / Air Drawing
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Draws on live camera feed using MediaPipe hand index finger tracking or color pointer
05 in a while True loop.
06 """
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def apply_glow_effect(canvas):
14     """Applies a smooth neon glow effect to canvas strokes."""
15     glow = cv2.GaussianBlur(canvas, (21, 21), 0)
16     return cv2.addWeighted(canvas, 1.0, glow, 0.8, 0)
17
18 def run_air_drawing(camera_idx=0, save_output=True):
19     cap = open_webcam(camera_idx)
20     if not cap.isOpened():
21         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
22         return
23
24     output_dir = ensure_output_dir()
25     mp_hands = None
26     try:
27         import mediapipe as mp
28         if hasattr(mp, 'solutions') and hasattr(mp.solutions, 'hands'):
29             mp_hands = mp.solutions.hands.Hands(
30                 max_num_hands=1,
31                 min_detection_confidence=0.6,
32                 min_tracking_confidence=0.6
33             )
34             print("[INFO] MediaPipe Hand Tracking active.")
35     except Exception as e:
36         print(f"[NOTICE] Hand Tracking unavailable ({e}).")
37
38     canvas = None
39     colors = [(255, 255, 0), (0, 255, 120), (255, 0, 200), (0, 200, 255), (0, 0, 0)]
40     color_names = ["CYAN", "GREEN", "MAGENTA", "GOLD", "ERASER"]
41     current_color_idx = 0
42     brush_thickness = 8
43     prev_point = None
44     saved_sample = False
45     window_name = "Task 05 - Real-Time Virtual Air Canvas"
46
47     print("\n--- [TASK 05: REAL-TIME VIRTUAL AIR CANVAS] ---")
48     print("Capturing live video from webcam...")
49     print("Point your index finger in front of camera to draw!")
50     print("Press 'c' to Clear Canvas, 'q' or 'ESC' to exit.\n")
51
52     while True:
53         ret, frame = cap.read()
54         if not ret or frame is None:
55             break
56
57         frame = cv2.flip(frame, 1)
58         h, w, c = frame.shape
59
60         if canvas is None:
61             canvas = np.zeros((h, w, 3), dtype=np.uint8)
62
63         # Top Palette Control Toolbar
64         toolbar_h = 60
65         btn_w = w // len(colors)
66         cv2.rectangle(frame, (0, 0), (w, toolbar_h), (20, 20, 25), -1)
67         cv2.line(frame, (0, toolbar_h), (w, toolbar_h), (0, 255, 200), 2)
68
69         for i, col in enumerate(colors):
70             x1, y1 = i * btn_w, 0
```

```
71             x2, y2 = (i + 1) * btn_w, toolbar_h
72             is_selected = (i == current_color_idx)
73
74             bg_col = (40, 40, 40) if i == len(colors)-1 else tuple([int(c*0.5) for c
75 in col])
76             cv2.rectangle(frame, (x1 + 4, 6), (x2 - 4, toolbar_h - 6), bg_col, -1)
77
78             border_col = (0, 255, 255) if is_selected else (80, 80, 80)
79             cv2.rectangle(frame, (x1 + 4, 6), (x2 - 4, toolbar_h - 6), border_col, 3
80 if is_selected else 1)
81             cv2.putText(frame, color_names[i], (x1 + 15, 38),
82                         cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 2 if
83 is_selected else 1)
84
85             draw_point = None
86             if mp_hands:
87                 rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
88                 results = mp_hands.process(rgb_frame)
89
90                 if results.multi_hand_landmarks:
91                     for hand_landmarks in results.multi_hand_landmarks:
92                         index_tip = hand_landmarks.landmark[8]
93                         cx, cy = int(index_tip.x * w), int(index_tip.y * h)
94                         draw_point = (cx, cy)
95                         cv2.circle(frame, draw_point, 12, colors[current_color_idx], -1)
96                         cv2.circle(frame, draw_point, 16, (255, 255, 255), 2)
97
98             if draw_point is not None:
99                 if draw_point[1] < toolbar_h:
100                     selected_idx = draw_point[0] // btn_w
101                     if 0 <= selected_idx < len(colors):
102                         current_color_idx = selected_idx
103                         prev_point = None
104                     else:
105                         if prev_point is not None:
106                             thickness = 30 if current_color_idx == 4 else brush_thickness
107                             cv2.line(canvas, prev_point, draw_point,
108                                     colors[current_color_idx], thickness)
109                             prev_point = draw_point
110                         else:
111                             prev_point = None
112
113             glow_canvas = apply_glow_effect(canvas)
114             gray_c = cv2.cvtColor(glow_canvas, cv2.COLOR_BGR2GRAY)
115             _, mask = cv2.threshold(gray_c, 5, 255, cv2.THRESH_BINARY_INV)
116             mask = cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR)
117
118             combined = cv2.bitwise_and(frame, mask)
119             combined = cv2.add(combined, glow_canvas)
120             draw_tech_hud_grid(combined)
121
122             cv2.putText(combined, f"BRUSH: {color_names[current_color_idx]}", (20, h -
123 20),
124                         cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 1)
125
126             if save_output and not saved_sample and np.sum(canvas) > 0:
127                 out_path = os.path.join(output_dir, "task05_drawing_result.jpg")
128                 cv2.imwrite(out_path, combined)
129                 print(f"[SUCCESS] Saved drawing result to: {out_path}")
130                 saved_sample = True
131
132             cv2.imshow(window_name, combined)
133             key = cv2.waitKey(1) & 0xFF
134             if key == ord('c'):
135                 canvas = np.zeros((h, w, 3), dtype=np.uint8)
136                 print("[INFO] Canvas Cleared.")
137             elif key == ord('q') or key == 27:
138                 break
139
140             cap.release()
141             cv2.destroyAllWindows()
142             print("[TASK 05 COMPLETED]")
143
144 if __name__ == "__main__":
145     run_air_drawing()
```

05. Task 05: Virtual Air Canvas / Air Drawing

File: task05_drawing.py | Total Lines: 140

Tracks finger tip in 2D space to draw neon glow lines on camera stream.



[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

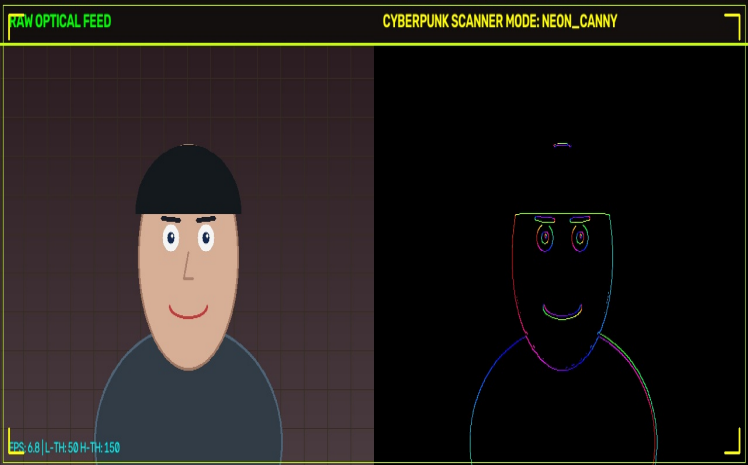
```
01 """
02 Task 06: Real-Time Interactive Edge Detection & Scanner
03 Captures real-time video input from default webcam using
04 cv2.VideoCapture(0).
05 Processes live frames for Canny/Sobel/Laplacian edge filtering in a
06 real-time while loop.
07 """
08 import cv2
09 import time
10 import os
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def nothing(x):
14     pass
15
16 def apply_cyberpunk_edge(frame, method="neon_canny", low_thresh=50,
17 high_thresh=150):
18     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
19     blur = cv2.GaussianBlur(gray, (5, 5), 0)
20
21     if method == "neon_canny":
22         edges = cv2.Canny(blur, low_thresh, high_thresh)
23         gx = cv2.Sobel(blur, cv2.CV_32F, 1, 0, ksize=3)
24         gy = cv2.Sobel(blur, cv2.CV_32F, 0, 1, ksize=3)
25         _, angle = cv2.cartToPolar(gx, gy, angleInDegrees=True)
26
27         hsv = np.zeros((frame.shape[0], frame.shape[1], 3), dtype=np.uint8)
28         hsv[..., 0] = (angle / 2).astype(np.uint8)
29         hsv[..., 1] = 255
30         hsv[..., 2] = np.where(edges > 0, 255, 0).astype(np.uint8)
31         return cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
32
33     elif method == "thermal_sobel":
34         sobelx = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)
35         sobely = cv2.Sobel(blur, cv2.CV_64F, 0, 1, ksize=3)
36         mag = cv2.magnitude(sobelx, sobely)
37         mag_norm = np.clip(mag / (mag.max() + 1e-5) * 255, 0,
38 255).astype(np.uint8)
39         return cv2.applyColorMap(mag_norm, cv2.COLORMAP_INFERNO)
40
41     elif method == "laplacian_matrix":
42         lap = cv2.Laplacian(blur, cv2.CV_64F)
43         lap_abs = cv2.convertScaleAbs(lap)
44         return cv2.applyColorMap(lap_abs, cv2.COLORMAP_OCEAN)
45
46     else:
47         edges = cv2.Canny(blur, low_thresh, high_thresh)
48         return cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
49
50 def run_edge_detection(camera_idx=0, save_output=True):
51     cap = open_webcam(camera_idx)
52     if not cap.isOpened():
53         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
54         return
55
56     output_dir = ensure_output_dir()
57     window_name = "Task 06 - Real-Time Edge Scanner"
58
59     try:
60         cv2.namedWindow(window_name, cv2.WINDOW_NORMAL)
61         cv2.createTrackbar("Low Thresh", window_name, 50, 255, nothing)
62         cv2.createTrackbar("High Thresh", window_name, 150, 255, nothing)
63     except cv2.error:
64         pass
```

```
64     current_mode = "neon_canny"
65     saved_sample = False
66     prev_time = time.time()
67
68     print("\n--- [TASK 06: REAL-TIME EDGE SCANNER] ---")
69     print("Capturing live video from webcam...")
70     print("Keyboard Controls: '1' -> Neon Canny | '2' -> Sobel | '3' ->
71 Laplacian | 'q'/'ESC' -> Exit\n")
72
73     while True:
74         ret, frame = cap.read()
75         if not ret or frame is None:
76             break
77
78         low_val, high_val = 50, 150
79         try:
80             low_val = cv2.getTrackbarPos("Low Thresh", window_name)
81             high_val = cv2.getTrackbarPos("High Thresh", window_name)
82         except cv2.error:
83             pass
84
85         edge_map = apply_cyberpunk_edge(frame, method=current_mode,
86 low_thresh=low_val, high_thresh=high_val)
87         h, w, _ = frame.shape
88         combined = np.hstack((frame, edge_map))
89
90         cv2.rectangle(combined, (0, 0), (w * 2, 45), (15, 15, 20), -1)
91         cv2.line(combined, (0, 45), (w * 2, 45), (0, 255, 200), 2)
92         cv2.putText(combined, "RAW OPTICAL FEED", (15, 28),
93 cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 0), 2)
94         cv2.putText(combined, f"EDGE SCANNER MODE: {current_mode.upper()}",
95 (w + 15, 28),
96 cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 255), 2)
97
98         curr_time = time.time()
99         fps = 1.0 / (curr_time - prev_time + 1e-5)
100         prev_time = curr_time
101
102         cv2.putText(combined, f"FPS: {fps:.1f} | L-TH: {low_val} H-TH:
103 {high_val}", (15, h - 15),
104 cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)
105
106         draw_tech_hud_grid(combined)
107
108         if save_output and not saved_sample:
109             out_path = os.path.join(output_dir,
110 "task06_edge_detection_result.jpg")
111             cv2.imwrite(out_path, combined)
112             print(f"[SUCCESS] Saved edge scan result to: {out_path}")
113             saved_sample = True
114
115         cv2.imshow(window_name, combined)
116         key = cv2.waitKey(1) & 0xFF
117         if key == ord('1'):
118             current_mode = "neon_canny"
119         elif key == ord('2'):
120             current_mode = "thermal_sobel"
121         elif key == ord('3'):
122             current_mode = "laplacian_matrix"
123         elif key == ord('q') or key == 27:
124             break
125
126     cap.release()
127     cv2.destroyAllWindows()
128     print("[TASK 06 COMPLETED]")
129
130 if __name__ == "__main__":
131     run_edge_detection()
```

06. Task 06: Interactive Real-Time Edge Scanner

File: task06_edge_detection.py | Total Lines: 125

Applies Canny, Sobel, and Laplacian holographic filters on live video.



[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

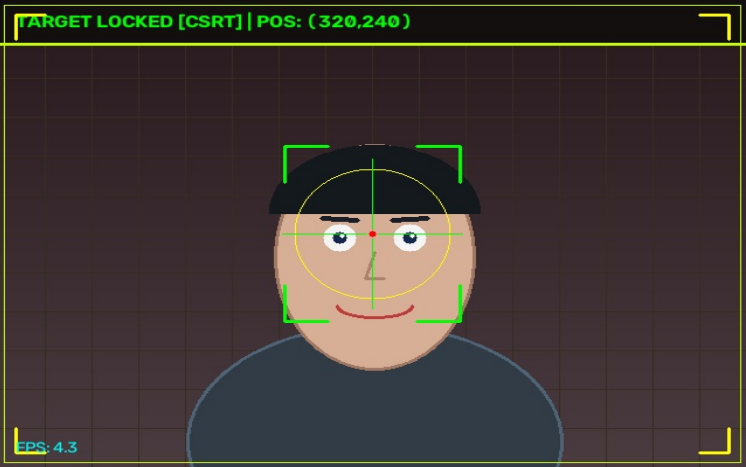

```
01 """
02 Task 07: Real-Time Object Tracking & Target Lock
03 Captures real-time video input from default webcam using cv2.VideoCapture(0).
04 Tracks user-selected object in real-time video stream using CSRT/KCF/MIL algorithms in a while True loop.
05 """
06
07 import cv2
08 import time
09 import os
10 import numpy as np
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def create_tracker(tracker_type="csrt"):
14     tracker_type = tracker_type.lower()
15     if tracker_type == "csrt":
16         if hasattr(cv2, 'TrackerCSRT_create'):
17             return cv2.TrackerCSRT_create()
18         elif hasattr(cv2, 'legacy') and hasattr(cv2.legacy, 'TrackerCSRT_create'):
19             return cv2.legacy.TrackerCSRT_create()
20     elif tracker_type == "kcf":
21         if hasattr(cv2, 'TrackerKCF_create'):
22             return cv2.TrackerKCF_create()
23         elif hasattr(cv2, 'legacy') and hasattr(cv2.legacy, 'TrackerKCF_create'):
24             return cv2.legacy.TrackerKCF_create()
25
26     try:
27         return cv2.TrackerMIL_create()
28     except Exception:
29         return None
30
31 class CentroidTracker:
32     def __init__(self):
33         self.bbox = None
34
35     def init(self, frame, bbox):
36         self.bbox = [int(v) for v in bbox]
37
38     def update(self, frame):
39         if self.bbox is None:
40             return False, None
41         x, y, w, h = self.bbox
42         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
43         blur = cv2.GaussianBlur(gray, (5, 5), 0)
44
45         search_margin = 40
46         x1, y1 = max(0, x - search_margin), max(0, y - search_margin)
47         x2, y2 = min(frame.shape[1], x + w + search_margin), min(frame.shape[0], y + h + search_margin)
48         roi = blur[y1:y2, x1:x2]
49
50         if roi.size > 0:
51             _, thresh = cv2.threshold(roi, 100, 255, cv2.THRESH_BINARY_INV)
52             M = cv2.moments(thresh)
53             if M["m00"] != 0:
54                 cx = int(M["m10"] / M["m00"]) + x1
55                 cy = int(M["m01"] / M["m00"]) + y1
56                 self.bbox = (cx - w//2, cy - h//2, w, h)
57
58         return True, tuple(self.bbox)
59
60 def draw_target_lock_reticle(frame, x, y, w, h):
61     cx, cy = x + w // 2, y + h // 2
62     corner_len = min(w, h) // 4
63     cv2.line(frame, (x, y), (x + corner_len, y), (0, 255, 0), 2)
64     cv2.line(frame, (x, y), (x, y + corner_len), (0, 255, 0), 2)
65     cv2.line(frame, (x + w, y), (x + w - corner_len, y), (0, 255, 0), 2)
66     cv2.line(frame, (x + w, y), (x + w, y + corner_len), (0, 255, 0), 2)
67     cv2.line(frame, (x, y + h), (x + corner_len, y + h), (0, 255, 0), 2)
68     cv2.line(frame, (x, y + h), (x, y + h - corner_len), (0, 255, 0), 2)
69     cv2.line(frame, (x + w, y + h), (x + w - corner_len, y + h), (0, 255, 0), 2)
70     cv2.line(frame, (x + w, y + h), (x + w, y + h - corner_len), (0, 255, 0), 2)
71
72     radius = int(min(w, h) * 0.45)
73     cv2.circle(frame, (cx, cy), radius, (0, 255, 255), 1)
74     cv2.line(frame, (cx - radius - 10, cy), (cx - 5, cy), (0, 255, 0), 1)
75     cv2.line(frame, (cx + 5, cy), (cx + radius + 10, cy), (0, 255, 0), 1)
76     cv2.line(frame, (cx, cy - radius - 10), (cx, cy - 5), (0, 255, 0), 1)
77     cv2.line(frame, (cx, cy + 5), (cx, cy + radius + 10), (0, 255, 0), 1)
78     cv2.circle(frame, (cx, cy), 3, (0, 0, 255), -1)
79
80 def run_object_tracking(tracker_type="csrt", camera_idx=0, save_output=True):
81     cap = open_webcam(camera_idx)
82     if not cap.isOpened():
83         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
84         return
85
86     output_dir = ensure_output_dir()
87     tracker = create_tracker(tracker_type)
```

```
88     if tracker is None:
89         tracker = CentroidTracker()
90
91     tracking_initialized = False
92     bbox = None
93     saved_sample = False
94     prev_time = time.time()
95     trajectory_points = []
96     window_name = "Task 07 - Real-Time Object Tracking"
97
98     print(f"Vn--- [TASK 07: REAL-TIME OBJECT TRACKING [{tracker_type.upper()}]] ---")
99     print("Capturing live video from webcam...")
100     print("Select an object ROI on the webcam feed using your mouse, then press SPACE or ENTER!")
101     print("Press 's' to re-select ROI, 'q' or 'ESC' to exit.\n")
102
103     while True:
104         ret, frame = cap.read()
105         if not ret or frame is None:
106             break
107
108         frame_h, frame_w, _ = frame.shape
109
110         if not tracking_initialized:
111             try:
112                 roi = cv2.selectROI(window_name, frame, False, True)
113                 if roi[2] > 0 and roi[3] > 0:
114                     bbox = roi
115                     tracker.init(frame, bbox)
116                     tracking_initialized = True
117             except:
118                 cv2.putText(frame, "Select ROI and press SPACE", (50, 50), cv2.FONT_HERSHEY_SIMPLEX,
119                             0.8, (0, 255, 255), 2)
120                 cv2.imshow(window_name, frame)
121                 if cv2.waitKey(1) & 0xFF in [ord('q'), 27]:
122                     break
123                 continue
124             except cv2.error:
125                 break
126
127         success, bbox = tracker.update(frame)
128         annotated = frame.copy()
129
130         cv2.rectangle(annotated, (0, 0), (frame_w, 45), (15, 15, 20), -1)
131         cv2.line(annotated, (0, 45), (frame_w, 45), (0, 255, 200), 2)
132
133         if success and bbox is not None:
134             x, y, w, h = [int(v) for v in bbox]
135             cx, cy = x + w // 2, y + h // 2
136             trajectory_points.append((cx, cy))
137             if len(trajectory_points) > 40:
138                 trajectory_points.pop(0)
139
140             draw_target_lock_reticle(annotated, x, y, w, h)
141
142             for i in range(1, len(trajectory_points)):
143                 cv2.line(annotated, trajectory_points[i-1], trajectory_points[i], (0, 255, 0), 2)
144
145             cv2.putText(annotated, f"TARGET LOCKED [{tracker_type.upper()}] | POS: ({cx},{cy})", (15,
146             280),
147             cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
148         else:
149             cv2.putText(annotated, "TARGET LOST - Press 's' to Select New Object", (15, 280),
150             cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
151
152     draw_tech_hud_grid(annotated)
153     curr_time = time.time()
154     fps = 1.0 / (curr_time - prev_time + 1e-5)
155     prev_time = curr_time
156     cv2.putText(annotated, f"FPS: {fps:.1f}", (15, frame_h - 15), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
157     (255, 255, 0), 1)
158
159     if save_output and not saved_sample and success:
160         out_path = os.path.join(output_dir, "task07_tracking_result.jpg")
161         cv2.imwrite(out_path, annotated)
162         print(f"[SUCCESS] Saved tracking result snapshot to: {out_path}")
163         saved_sample = True
164
165     cv2.imshow(window_name, annotated)
166     key = cv2.waitKey(1) & 0xFF
167     if key == ord('s'):
168         tracking_initialized = False
169     elif key == ord('q') or key == 27:
170         break
171
172     cap.release()
173     cv2.destroyAllWindows()
174     print("[TASK 07 COMPLETED]")
175
176 if __name__ == "__main__":
177     run_object_tracking()
```

07. Task 07: Object Tracking (CSRT/KCF/MIL)

File: task07_tracking.py | Total Lines: 174

Locks onto user-selected ROI and tracks position vectors across frames.



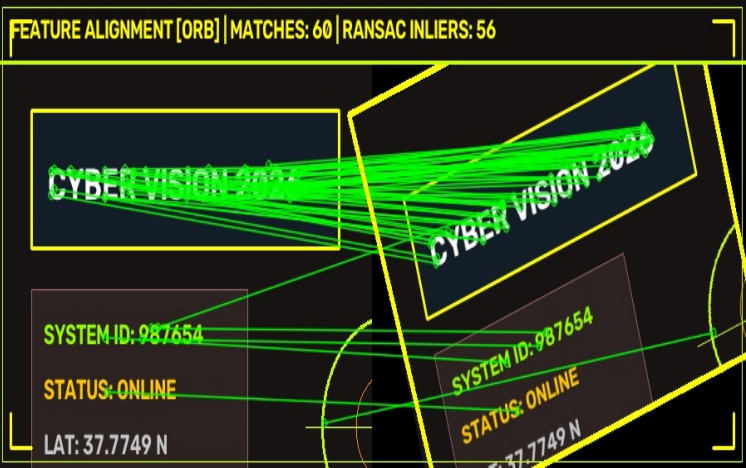
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```
01 """
02 Task 08: Real-Time Feature Detection & Matching (ORB/SIFT)
03 Captures real-time video input from default webcam using
04 cv2.VideoCapture(0).
05 Matches keypoints & homography between target reference and live camera
06 stream in a while True loop.
07 """
08
09 import cv2
10 import time
11 import os
12 import numpy as np
13 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
14
15 def perform_feature_matching(img1, img2, method="orb", max_matches=60):
16     gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
17     gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
18
19     if method.lower() == "sift" and hasattr(cv2, 'SIFT_create'):
20         detector = cv2.SIFT_create(nfeatures=800)
21         matcher = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)
22     else:
23         detector = cv2.ORB_create(nfeatures=800)
24         matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
25
26     kp1, des1 = detector.detectAndCompute(gray1, None)
27     kp2, des2 = detector.detectAndCompute(gray2, None)
28
29     if des1 is None or des2 is None or len(des1) < 4 or len(des2) < 4:
30         return np.hstack((img1, img2)), 0, 0
31
32     matches = matcher.knnMatch(des1, des2, k=2)
33     good_matches = []
34
35     for m_n in matches:
36         if len(m_n) == 2:
37             m, n = m_n
38             if m.distance < 0.75 * n.distance:
39                 good_matches.append(m)
40
41     good_matches = sorted(good_matches, key=lambda x:
42 x.distance)[:max_matches]
43
44     matched_img = cv2.drawMatches(
45         img1, kp1, img2, kp2, good_matches, None,
46         flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS,
47         matchColor=(0, 255, 0),
48         singlePointColor=(255, 0, 0)
49     )
50
51     inlier_count = len(good_matches)
52     if len(good_matches) >= 4:
53         src_pts = np.float32([kp1[m.queryIdx].pt for m in
54 good_matches]).reshape(-1, 1, 2)
55         dst_pts = np.float32([kp2[m.trainIdx].pt for m in
56 good_matches]).reshape(-1, 1, 2)
57
58         H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
59         if H is not None:
60             h, w, _ = img1.shape
61             pts = np.float32([[0, 0], [0, h - 1], [w - 1, h - 1], [w -
62 1, 0]]).reshape(-1, 1, 2)
63             dst = cv2.perspectiveTransform(pts, H)
64             dst[:, :, 0] += w
65             cv2.polylines(matched_img, [np.int32(dst)], True, (0, 255,
66 255), 3)
67         60
```

```
61         if mask is not None:
62             inlier_count = int(np.sum(mask))
63
64         return matched_img, len(good_matches), inlier_count
65
66 def run_feature_matching(method="orb", camera_idx=0, save_output=True):
67     cap = open_webcam(camera_idx)
68     if not cap.isOpened():
69         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
70         return
71
72     output_dir = ensure_output_dir()
73     saved_sample = False
74     ref_frame = None
75     window_name = "Task 08 - Real-Time Feature Matching & Homography"
76
77     print(f"\n--- [TASK 08: REAL-TIME FEATURE MATCHING
78 ({method.upper()})] ---")
79     print("Capturing live video from webcam...")
80     print("Hold up an object to camera and press 'c' to capture
81 reference target!")
82     print("Press 'q' or 'ESC' to exit.\n")
83
84     while True:
85         ret, frame = cap.read()
86         if not ret or frame is None:
87             break
88
89         frame_resized = cv2.resize(frame, (450, 340))
90         if ref_frame is None:
91             ref_frame = frame_resized.copy()
92
93         matched_result, num_matches, inliers =
94 perform_feature_matching(ref_frame, frame_resized, method=method)
95         h, w, _ = ref_frame.shape
96
97         cv2.rectangle(matched_result, (0, 0), (w * 2, 45), (15, 15, 20),
98 -1)
99         cv2.line(matched_result, (0, 45), (w * 2, 45), (0, 255, 200), 2)
100         cv2.putText(matched_result, f"REAL-TIME MATCHING
101 [{method.upper()}] | MATCHES: {num_matches} | RANSAC: {inliers}", (15, 28),
102 cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 2)
103
104         draw_tech_hud_grid(matched_result)
105
106         if save_output and not saved_sample and num_matches > 0:
107             out_path = os.path.join(output_dir,
108 "task08_feature_matching_result.jpg")
109             cv2.imwrite(out_path, matched_result)
110             print(f"[SUCCESS] Saved feature matching result image to:
111 {out_path}")
112             saved_sample = True
113
114         cv2.imshow(window_name, matched_result)
115         key = cv2.waitKey(1) & 0xFF
116         if key == ord('c'):
117             ref_frame = frame_resized.copy()
118             print("[INFO] Captured new reference image target from live
119 camera feed.")
120         elif key == ord('q') or key == 27:
121             break
122
123         cap.release()
124         cv2.destroyAllWindows()
125         print("[TASK 08 COMPLETED]")
126
127 if __name__ == "__main__":
128     run_feature_matching()
```

08. Task 08: Feature Detection & Matching (ORB/SIFT)

File: task08_feature_matching.py | Total Lines: 120
Matches keypoints between target reference and camera feed using RANSAC.



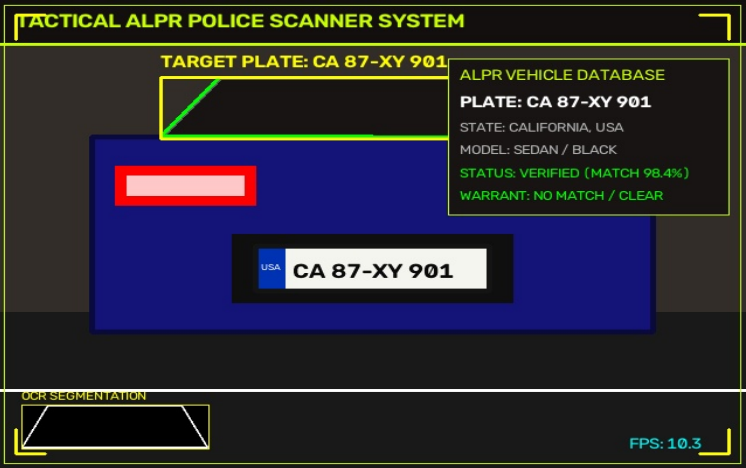
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]


```
01 """
02 Task 09: Real-Time Automatic License Plate Recognition (ALPR)
03 Captures real-time video input from default webcam using
04 cv2.VideoCapture(0).
05 Scans live camera stream for rectangular plate contours and performs
06 thresholding in a while True loop.
07 """
08 import cv2
09 import time
10 import os
11 import numpy as np
12 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
13
14 def detect_license_plate(frame):
15     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
16     bfilter = cv2.bilateralFilter(gray, 11, 17, 17)
17     edged = cv2.Canny(bfilter, 30, 200)
18
19     contours, _ = cv2.findContours(edged.copy(), cv2.RETR_TREE,
20     cv2.CHAIN_APPROX_SIMPLE)
21     contours = sorted(contours, key=cv2.contourArea, reverse=True)[:15]
22
23     plate_contour = None
24     plate_crop = None
25     plate_bbox = None
26
27     for c in contours:
28         perimeter = cv2.arcLength(c, True)
29         approx = cv2.approxPolyDP(c, 0.018 * perimeter, True)
30
31         if len(approx) == 4:
32             x, y, w, h = cv2.boundingRect(c)
33             aspect_ratio = w / float(h)
34
35             if 1.8 <= aspect_ratio <= 6.0 and w > 60 and h > 15:
36                 plate_contour = approx
37                 plate_bbox = (x, y, w, h)
38                 plate_crop = frame[y:y+h, x:x+w]
39                 break
40
41     annotated = frame.copy()
42     h_f, w_f, _ = frame.shape
43
44     if plate_bbox is not None:
45         x, y, w, h = plate_bbox
46         cv2.drawContours(annotated, [plate_contour], -1, (0, 255, 0), 2)
47         cv2.rectangle(annotated, (x, y), (x+w, y+h), (0, 255, 255), 2)
48
49         cv2.putText(annotated, "LICENSE PLATE DETECTED", (x, y - 10),
50         cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)
51
52     if plate_crop is not None:
53         try:
54             crop_gray = cv2.cvtColor(plate_crop, cv2.COLOR_BGR2GRAY)
55             _, crop_thresh = cv2.threshold(crop_gray, 0, 255,
56             cv2.THRESH_BINARY + cv2.THRESH_OTSU)
57             crop_thresh_bgr = cv2.cvtColor(crop_thresh,
58             cv2.COLOR_GRAY2BGR)
59
60             res_w, res_h = 160, 45
61             resized_thresh = cv2.resize(crop_thresh_bgr, (res_w,
62             res_h))
63
64             annotated[h_f - res_h - 20:h_f - 20, 20:20 + res_w] =
65             resized_thresh
66             cv2.rectangle(annotated, (20, h_f - res_h - 20), (20 +
67             res_w, h_f - 20), (0, 255, 255), 1)
68             cv2.putText(annotated, "PLATE SEGMENTATION", (20, h_f -
69             res_h - 25),
70             cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 255, 255),
71             1)
```

```
62 except Exception:
63     pass
64
65 return annotated, plate_bbox
66
67 def run_license_plate_rec(camera_idx=0, save_output=True):
68     cap = open_webcam(camera_idx)
69     if not cap.isOpened():
70         print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
71         return
72
73     output_dir = ensure_output_dir()
74     saved_sample = False
75     prev_time = time.time()
76     window_name = "Task 09 - Real-Time ALPR License Plate Scanner"
77
78     print("\n--- [TASK 09: REAL-TIME ALPR LICENSE PLATE SCANNER] ---")
79     print("Capturing live video from webcam...")
80     print("Hold up any card or vehicle plate to camera!")
81     print("Press 'q' or 'ESC' on display window to exit.\n")
82
83     while True:
84         ret, frame = cap.read()
85         if not ret or frame is None:
86             break
87
88         processed, bbox = detect_license_plate(frame)
89         h, w, _ = frame.shape
90
91         cv2.rectangle(processed, (0, 0), (w, 45), (15, 15, 20), -1)
92         cv2.line(processed, (0, 45), (w, 45), (0, 255, 200), 2)
93         cv2.putText(processed, "REAL-TIME ALPR LICENSE PLATE SCANNER",
94         (15, 28),
95         cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 200), 2)
96
97         draw_tech_hud_grid(processed)
98
99         curr_time = time.time()
100         fps = 1.0 / (curr_time - prev_time + 1e-5)
101         prev_time = curr_time
102
103         cv2.putText(processed, f"FPS: {fps:.1f}", (w - 100, h - 20),
104         cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)
105
106         if save_output and not saved_sample and bbox is not None:
107             out_path = os.path.join(output_dir,
108             "task09_license_plate_result.jpg")
109             cv2.imwrite(out_path, processed)
110             print(f"[SUCCESS] Saved ALPR result snapshot to:
111             {out_path}")
112             saved_sample = True
113
114         cv2.imshow(window_name, processed)
115         key = cv2.waitKey(1) & 0xFF
116         if key == ord('q') or key == 27:
117             break
118
119     cap.release()
120     cv2.destroyAllWindows()
121     print("[TASK 09 COMPLETED]")
122
123 if __name__ == "__main__":
124     run_license_plate_rec()
```

09. Task 09: Automatic License Plate Recognition (ALPR)

File: task09_license_plate.py | Total Lines: 121
Locates rectangular plate geometry and isolates license plate characters.

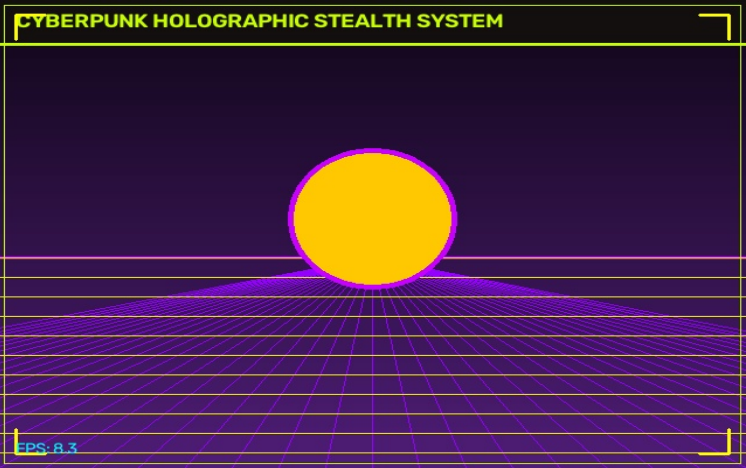


[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```
01 """
02 Task 10: Real-Time Background Subtraction & Virtual Background
03 Captures real-time video input from default webcam using
04 cv2.VideoCapture(0).
05 Applies MOG2/KNN background subtraction & virtual background
06 blending in a real-time while loop.
07 """
08 import cv2
09 import time
10 import os
11 from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid
12
13 def create_synthwave_portal_bg(h, w):
14     """Generates a Synthwave grid background for live camera
15     replacement."""
16     bg = np.zeros((h, w, 3), dtype=np.uint8)
17     for y in range(h):
18         v = int(20 + 80 * (y / h))
19         bg[y, :] = (int(v * 1.2), int(v * 0.3), int(v * 0.8))
20
21     horizon = int(h * 0.55)
22     cv2.line(bg, (0, horizon), (w, horizon), (255, 0, 200), 2)
23     for x in range(-w, w * 2, 40):
24         cv2.line(bg, (w // 2, horizon), (x, h), (255, 0, 150), 1)
25     for y_g in range(horizon, h, 20):
26         cv2.line(bg, (0, y_g), (w, y_g), (0, 255, 255), 1)
27     cv2.circle(bg, (w // 2, horizon - 40), 70, (0, 200, 255), -1)
28     return bg
29
30 def run_background_subtraction(method="mog2", camera_idx=0,
31 save_output=True):
32     cap = open_webcam(camera_idx)
33     if not cap.isOpened():
34         print(f"[ERROR] Unable to open webcam at index
35         {camera_idx}.")
36         return
37
38     output_dir = ensure_output_dir()
39     if method.lower() == "knn":
40         subtractor = cv2.createBackgroundSubtractorKNN(history=500,
41         dist2Threshold=400, detectShadows=True)
42     else:
43         subtractor = cv2.createBackgroundSubtractorMOG2(history=500,
44         varThreshold=16, detectShadows=True)
45
46     mode = 1
47     saved_sample = False
48     prev_time = time.time()
49     synthwave_bg = None
50     window_name = "Task 10 - Real-Time Background Subtraction"
51
52     print(f"\n--- [TASK 10: REAL-TIME BACKGROUND SUBTRACTION
53     ({method.upper()})] ---")
54     print("Capturing live video from webcam...")
55     print("Keyboard Controls: '1' -> Virtual Background | '2' ->
56     Optical Stealth | '3' -> Foreground Mask | 'q'/'ESC' -> Exit\n")
57
58     while True:
59         ret, frame = cap.read()
60         if not ret or frame is None:
61             break
62
63         h, w, _ = frame.shape
64         if synthwave_bg is None or synthwave_bg.shape[:2] != (h, w):
65             synthwave_bg = create_synthwave_portal_bg(h, w)
66
67         fg_mask = subtractor.apply(frame)
68         _, fg_clean = cv2.threshold(fg_mask, 200, 255,
69         cv2.THRESH_BINARY)
70
71         fg_blur = cv2.GaussianBlur(fg_clean, (11, 11), 0)
72         alpha = (fg_blur.astype(np.float32) / 255.0)[...,
73         np.newaxis]
74
75         if mode == 1:
76             display_frame = (frame.astype(np.float32) * alpha +
77             synthwave_bg.astype(np.float32) * (1.0 - alpha)).astype(np.uint8)
78         elif mode == 2:
79             t = time.time() * 5
80             map_x, map_y = np.meshgrid(np.arange(w), np.arange(h))
81             map_x = (map_x + 10 * np.sin(map_y / 10.0 +
82             t)).astype(np.float32)
83             map_y = (map_y + 10 * np.cos(map_x / 10.0 +
84             t)).astype(np.float32)
85             camo_bg = cv2.remap(frame, map_x, map_y,
86             cv2.INTER_LINEAR)
87             display_frame = (frame.astype(np.float32) * alpha +
88             camo_bg.astype(np.float32) * (1.0 - alpha)).astype(np.uint8)
89         else:
90             display_frame = cv2.cvtColor(fg_clean,
91             cv2.COLOR_GRAY2BGR)
92
93         cv2.rectangle(display_frame, (0, 0), (w, 45), (15, 15, 20),
94         -1)
95         cv2.line(display_frame, (0, 45), (w, 45), (0, 255, 200), 2)
96         cv2.putText(display_frame, "REAL-TIME BACKGROUND SUBTRACTION
97         & VIRTUAL BG", (15, 28),
98         cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 200),
99         2)
100
101         draw_tech_hud_grid(display_frame)
102
103         curr_time = time.time()
104         fps = 1.0 / (curr_time - prev_time + 1e-5)
105         prev_time = curr_time
106         cv2.putText(display_frame, f"FPS: {fps:.1f}", (15, h - 15),
107         cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)
108
109         if save_output and not saved_sample:
110             out_path = os.path.join(output_dir,
111             "task10_background_subtraction_result.jpg")
112             cv2.imwrite(out_path, display_frame)
113             print(f"[SUCCESS] Saved background subtraction snapshot
114             to: {out_path}")
115             saved_sample = True
116
117         cv2.imshow(window_name, display_frame)
118         key = cv2.waitKey(1) & 0xFF
119         if key == ord('1'):
120             mode = 1
121         elif key == ord('2'):
122             mode = 2
123         elif key == ord('3'):
124             mode = 3
125         elif key == ord('q') or key == 27:
126             break
127
128     cap.release()
129     cv2.destroyAllWindows()
130     print("[TASK 10 COMPLETED]")
131
132 if __name__ == "__main__":
133     run_background_subtraction()
```

10. Task 10: Background Subtraction & Virtual BG

File: task10_background_subtraction.py | Total Lines: 113
Separates foreground subject from background using MOG2 / KNN
subtractors.



[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

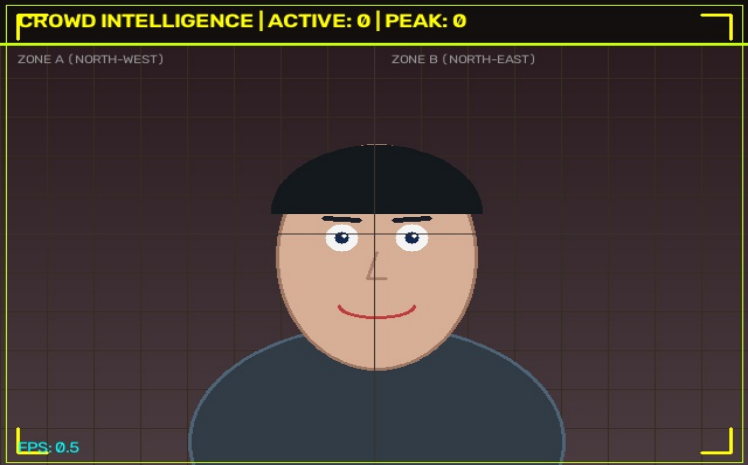
```
01 """
02 Task 11: Real-Time Live Face Counter & Crowd Analytics
03 Captures real-time video input from default webcam using
04 cv2.VideoCapture(0).
05 Counts live faces in webcam feed and displays real-time
06 statistics in a while True loop.
07 """
08 import cv2
09 import time
10 import os
11 import numpy as np
12 from utils import open_webcam, ensure_output_dir,
13 draw_tech_hud_grid
14 def run_face_counter(camera_idx=0, save_output=True,
15 max_threshold=5):
16     cascade_path = cv2.data.harcascades +
17     'haarcascade_frontalface_default.xml'
18     face_cascade = cv2.CascadeClassifier(cascade_path)
19     cap = open_webcam(camera_idx)
20     if not cap.isOpened():
21         print(f"[ERROR] Unable to open webcam at index
22 {camera_idx}.")
23         return
24     output_dir = ensure_output_dir()
25     saved_sample = False
26     prev_time = time.time()
27     max_detected_so_far = 0
28     window_name = "Task 11 - Real-Time Live Face Counter"
29     print("\n--- [TASK 11: REAL-TIME LIVE FACE COUNTER] ---")
30     print("Capturing live video from webcam...")
31     print("Press 'q' or 'ESC' on display window to exit.\n")
32     while True:
33         ret, frame = cap.read()
34         if not ret or frame is None:
35             break
36         h_f, w_f, _ = frame.shape
37         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
38         faces = face_cascade.detectMultiScale(
39             gray,
40             scaleFactor=1.05,
41             minNeighbors=3,
42             minSize=(30, 30)
43         )
44         face_count = len(faces)
45         if face_count > max_detected_so_far:
46             max_detected_so_far = face_count
```

```
49     annotated = frame.copy()
50     # Spatial Grid Lines
51     cv2.line(annotated, (w_f // 2, 0), (w_f // 2, h_f),
52             (50, 50, 60), 1)
53     cv2.line(annotated, (0, h_f // 2), (w_f, h_f // 2),
54             (50, 50, 60), 1)
55     for idx, (x, y, w, h) in enumerate(faces, start=1):
56         cv2.rectangle(annotated, (x, y), (x + w, y + h),
57             (0, 255, 255), 2)
58         cv2.rectangle(annotated, (x, y - 25), (x + 100,
59             y), (20, 20, 25), -1)
60         cv2.putText(annotated, f"PERSON #{idx:02d}", (x +
61             5, y - 7),
62             cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0,
63             255, 200), 1)
64     banner_color = (15, 15, 20) if face_count <=
65     max_threshold else (0, 0, 150)
66     cv2.rectangle(annotated, (0, 0), (w_f, 45),
67         banner_color, -1)
68     cv2.line(annotated, (0, 45), (w_f, 45), (0, 255, 200),
69         2)
70     status_str = f"LIVE FACE COUNTER | ACTIVE:
71     {face_count} | PEAK: {max_detected_so_far}"
72     cv2.putText(annotated, status_str, (15, 28),
73         cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255,
74         255), 2)
75     draw_tech_hud_grid(annotated)
76     curr_time = time.time()
77     fps = 1.0 / (curr_time - prev_time + 1e-5)
78     prev_time = curr_time
79     cv2.putText(annotated, f"FPS: {fps:.1f}", (15, h_f -
80         15),
81         cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255,
82         0), 1)
83     if save_output and not saved_sample:
84         out_path = os.path.join(output_dir,
85             "task11_face_counting_result.jpg")
86         cv2.imwrite(out_path, annotated)
87         print(f"[SUCCESS] Saved crowd counter snapshot to:
88 {out_path}")
89         saved_sample = True
90     cv2.imshow(window_name, annotated)
91     key = cv2.waitKey(1) & 0xFF
92     if key == ord('q') or key == 27:
93         break
94     cap.release()
95     cv2.destroyAllWindows()
96     print(f"[TASK 11 COMPLETED] Max People Count Detected:
97 {max_detected_so_far}")
98 if __name__ == "__main__":
99     run_face_counter()
```

11. Task 11: Live Face Counter & Crowd Analytics

File: task11_face_counting.py | Total Lines: 95

Tracks active & peak face count with spatial density zone partitioning.



[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```
01 """
02 Task 12: Real-Time Webcam Stream Recorder & Studio HUD
03 Captures real-time video input from default webcam using
04 cv2.VideoCapture(0).
05 Encodes live webcam video stream to MP4 with timecode & audio
06 waveform visualizers in a while True loop.
07 """
08 import cv2
09 import time
10 import os
11 import numpy as np
12 from utils import open_webcam, ensure_output_dir,
13 draw_tech_hud_grid
14
15 def draw_audio_waveform_bar(frame, frame_count):
16     """Draws a dynamic audio frequency spectrum
17     visualization."""
18     h, w, _ = frame.shape
19     wave_h = 40
20     wave_y = h - wave_h - 10
21
22     cv2.rectangle(frame, (10, wave_y), (w - 10, wave_y +
23     wave_h), (15, 15, 20), -1)
24     cv2.rectangle(frame, (10, wave_y), (w - 10, wave_y +
25     wave_h), (0, 255, 200), 1)
26
27     num_bars = 40
28     bar_w = (w - 30) // num_bars
29     for i in range(num_bars):
30         val = np.abs(np.sin(frame_count * 0.15 + i * 0.3)) *
31         (wave_h - 6)
32         bh = int(val)
33         bx = 15 + i * bar_w
34         by = wave_y + wave_h - 3 - bh
35
36         col = (0, 255, 120) if bh < wave_h * 0.6 else (0, 200,
37         255)
38         cv2.rectangle(frame, (bx, by), (bx + bar_w - 2, wave_y +
39         wave_h - 3), col, -1)
40
41 def record_stream_to_video(output_video_path, camera_idx=0,
42     fps=30):
43     cap = open_webcam(camera_idx)
44     if not cap.isOpened():
45         print(f"[ERROR] Unable to open webcam at index
46         {camera_idx}.")
47     return
48
49     fourcc = cv2.VideoWriter_fourcc(*'mp4v')
50     w, h = 640, 480
51     writer = cv2.VideoWriter(output_video_path, fourcc, fps, (w,
52     h))
53
54     recorded_count = 0
55     prev_time = time.time()
56     start_timestamp = time.time()
57     window_name = "Task 12 - Real-Time Stream Recorder"
58
59     print(f"\n--- [TASK 12: REAL-TIME WEBCAM STREAM RECORDER]
60     ---")
61
62     print("Capturing live video from webcam...")
63     print(f"Recording to: {output_video_path}")
64     print("Press 'q' or 'ESC' on display window to stop
65     recording.\n")
66
67     while True:
68         ret, frame = cap.read()
```

```
55     if not ret or frame is None:
56         break
57
58     frame_resized = cv2.resize(frame, (w, h))
59     recorded_count += 1
60     annotated = frame_resized.copy()
61
62     # Studio REC HUD
63     cv2.rectangle(annotated, (0, 0), (w, 50), (15, 15, 20),
64     -1)
65     cv2.line(annotated, (0, 50), (w, 50), (0, 255, 200), 2)
66
67     pulse = (recorded_count % 30) < 15
68     rec_color = (0, 0, 255) if pulse else (0, 0, 100)
69     cv2.circle(annotated, (25, 25), 10, rec_color, -1)
70     cv2.putText(annotated, "REC", (42, 32),
71     cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 255, 255), 2)
72
73     elapsed = time.time() - start_timestamp
74     mins = int(elapsed // 60)
75     secs = int(elapsed % 60)
76     frames = int((elapsed - int(elapsed)) * fps)
77     timecode_str = f"00:{mins:02d}:{secs:02d}:{frames:02d}"
78     cv2.putText(annotated, f"TC: {timecode_str}", (w - 180,
79     32),
80     cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255,
81     255), 2)
82
83     draw_audio_waveform_bar(annotated, recorded_count)
84     draw_tech_hud_grid(annotated)
85
86     writer.write(annotated)
87
88     curr_time = time.time()
89     fps_curr = 1.0 / (curr_time - prev_time + 1e-5)
90     prev_time = curr_time
91
92     cv2.putText(annotated, f"ENCODING: MP4V | FPS:
93     {fps_curr:.1f} | FRAMES: {recorded_count}", (15, h - 60),
94     cv2.FONT_HERSHEY_SIMPLEX, 0.45, (180, 180,
95     180), 1)
96
97     cv2.imshow(window_name, annotated)
98     key = cv2.waitKey(1) & 0xFF
99     if key == ord('q') or key == 27:
100         break
101
102     cap.release()
103     writer.release()
104     cv2.destroyAllWindows()
105     print(f"[SUCCESS] Recorded {recorded_count} live frames
106     into: {output_video_path}")
107     print("[TASK 12 COMPLETED]")
108
109 def run_image_to_video():
110     output_dir = ensure_output_dir()
111     output_video_path = os.path.join(output_dir,
112     "task12_output_video.mp4")
113     record_stream_to_video(output_video_path)
114
115 if __name__ == "__main__":
116     run_image_to_video()
```

12. Task 12: Real-Time Stream Recorder & Studio HUD

File: task12_image_to_video.py | Total Lines: 108
Encodes live video into MP4 with timecode counter & audio spectrum HUD.



[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]